

Menofia University — Faculty of Engineering

Digital IC Design Using Verilog HDL

**Universal Asynchronous Receiver-
Transmitter (UART)
RTL Implementation**

— Implemented with Verilog HDL —

Ahmed Ahmed Nageeb Elbermawy

Menofia University, Faculty of Engineering

Tool: Xilinx Vivado | Language: Verilog HDL

Table of Contents

1. Introduction

2. Basic Concepts of UART

- Asynchronous Communication
- UART Frame Format
- Baud Rate
- Parity Bit
- Stop Bits

3. Design Methodology

- 16× Oversampling Procedure
- Input Synchronization (2-FF Synchronizer)

4. Top-Level Design Diagram

5. Component Breakdown

- Baud Rate Generator
- UART Transmitter
- UART Receiver
- Top-Level Wrapper

6. ASMD Charts

- ASMD Chart of the UART Transmitter
- ASMD Chart of the UART Receiver

7. Testbenches & Simulation

- Transmitter Testbench
- Full UART Loopback Testbench

8. References

1. Introduction

UART (Universal Asynchronous Receiver-Transmitter) is one of the most fundamental serial communication protocols in digital systems. It enables data exchange between two devices over just two wires — TX (transmit) and RX (receive) — without requiring a shared clock signal. This makes it simple, low-cost, and compatible with virtually every microcontroller, FPGA, and embedded system.

This report documents the design and RTL implementation of a complete UART transceiver in Verilog HDL using Xilinx Vivado. The design covers the full UART frame format including start bit, data bits, optional parity, and stop bit. The receiver employs 16× oversampling to reliably recover serial data without a shared clock reference. A 2-FF input synchronizer on the RX line prevents metastability from propagating into the FSM — a critical requirement in any real-world digital design.

The design follows a modular, hierarchical approach. Each component is developed and verified independently through simulation testbenches before integration into the top-level wrapper. Key features of this implementation:

- Parameterized design — DBITS and SB_TICK are configurable at elaboration time
- Runtime-selectable parity — even, odd, or none via a 2-bit parity_select input
- Parity error detection — flags a mismatch between received and expected parity
- Frame error detection — flags a missing stop bit (stop bit sampled as logic 0)
- 2-FF input synchronizer on RX — prevents metastability propagation
- Clean RTL style — separate present-state and next-state always blocks throughout
- Two testbenches — standalone TX verification and full TX→RX loopback with automated PASS/FAIL output

2. Basic Concepts of UART

Asynchronous Communication

The defining characteristic of UART is its asynchronous operation — the transmitter and receiver share no common clock. Each device uses its own internal oscillator. Synchronization is achieved entirely through the frame structure: a start bit signals the beginning and stop bits mark the end. Both sides must be pre-configured to the same baud rate and format.

UART Frame Format

A standard UART frame consists of the following fields transmitted serially in order: an idle high line, a start bit (logic 0), the data bits (LSB first), an optional parity bit, and one or more stop bits (logic 1). Figure 1 illustrates this structure.

UART Frame Structure

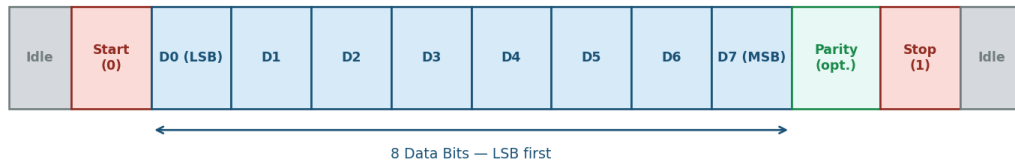


Figure 1. UART Frame Structure

Field	Logic Level	Description
Idle	1	Line rests high when no data is being sent
Start bit	0	Always logic low — signals the start of a new frame
Data bits	D0–D7	Sent LSB first; typically 8 bits
Parity bit	0 / 1	Optional error-detection bit (even, odd, or omitted)
Stop bit	1	Always logic high — marks the end of the frame

Baud Rate

The baud rate defines the number of bits transmitted per second (bps). Both devices must use identical baud rates; a mismatch results in corrupted data. With a 100 MHz system clock and 16× oversampling, the Baud Rate Generator counter limit is:

$$FINAL_VALUE = F_{clk} / (16 \times BaudRate) - 1$$

Baud Rate (bps)	FINAL_VALUE (100 MHz)	BRG_BITS needed
9 600	651	10
19 200	325	9
38 400	162	8
57 600	108	7
115 200	54	6

Parity Bit

An optional parity bit provides single-bit error detection. In even parity the parity bit is set so the total number of logic-1 bits (data + parity) is even; in odd parity the total is odd. If the receiver calculates a different parity than received, a parity_error flag is asserted.

Stop Bits

Stop bits are always logic high and mark the end of the frame. They provide the receiver time to process the received byte. This design supports a configurable stop-bit duration via the SB_TICK parameter (16 = 1 stop bit, 32 = 2 stop bits).

3. Design Methodology

UART Receiving Subsystem

In UART communication the transmitted signal does not include a clock, meaning the receiver has no explicit timing reference to know exactly when each bit begins and ends. Instead, both transmitter and receiver rely on a pre-agreed baud rate to stay synchronised in time.

16× Oversampling Procedure

To accurately recover the transmitted data, the receiver uses an oversampling technique. The Baud Rate Generator produces a sampling tick (s_tick) at 16 times the baud rate. The receiver counts these ticks to locate the centre of each bit — the point where the signal is most stable and least susceptible to noise or jitter.

The procedure for each received frame:

- 1. Monitor rx_sync in the idle state. A falling edge (logic 0) indicates the start of the start bit. Reset the tick counter.
- 2. Count 7 s_ticks — this positions sampling at the midpoint of the start bit (tick 7 of 16). Confirm the line is still low to reject noise glitches.
- 3. Count 16 s_ticks — now at the midpoint of data bit D0 (tick 15). Sample rx_sync and shift into the data register.
- 4. Repeat step 3 for the remaining 7 data bits.
- 5. If parity is enabled, repeat step 3 once to sample the parity bit.
- 6. Count through the stop bit period; verify rx_sync is high. Assert rx_done_tick.

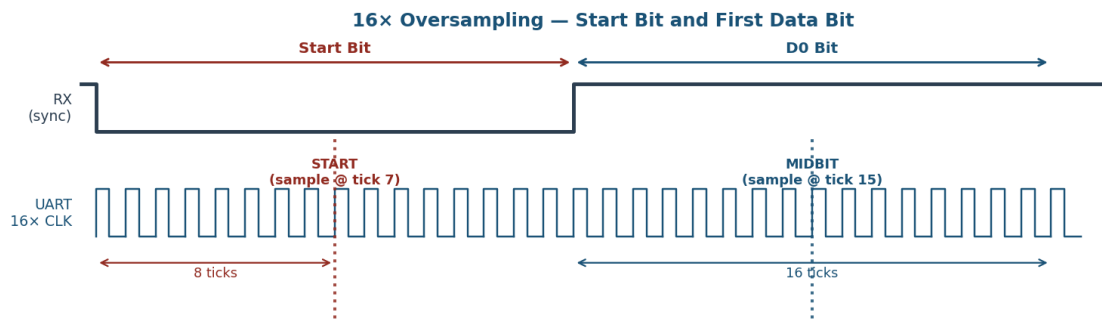


Figure 2. 16× Oversampling — Start Bit and First Data Bit

Input Synchronization (2-FF Synchronizer)

The rx line arrives from an external device and is therefore asynchronous to the FPGA's clock domain. Sampling it directly risks metastability — a flip-flop entering an undefined voltage state that can propagate corrupted values through the logic.

The solution is a two flip-flop (2-FF) synchronizer placed at the RX input before the FSM. The first FF may enter a metastable state, but it has a full clock period to resolve before the second FF (rx_sync) samples it. Both FFs reset to logic 1 (the UART idle level) to prevent a false start-bit detection on power-up.

2-FF Metastability Synchronizer on RX Input

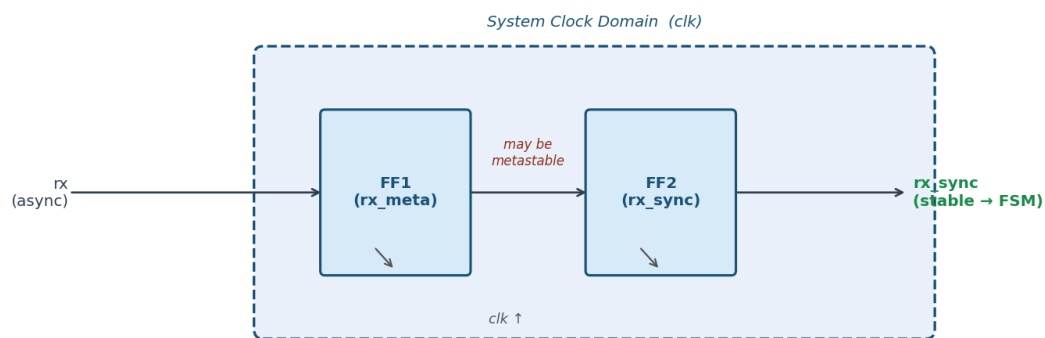


Figure 3. 2-FF Metastability Synchronizer on the RX Input

4. Top-Level Design Diagram

The system is composed of three primary blocks instantiated inside UART_top: the Baud Rate Generator, the Transmitter, and the Receiver. The Baud Rate Generator's done output is distributed to both TX and RX as the shared s_tick timing reference. The parity_select input is also shared, ensuring both sides always use the same parity configuration.

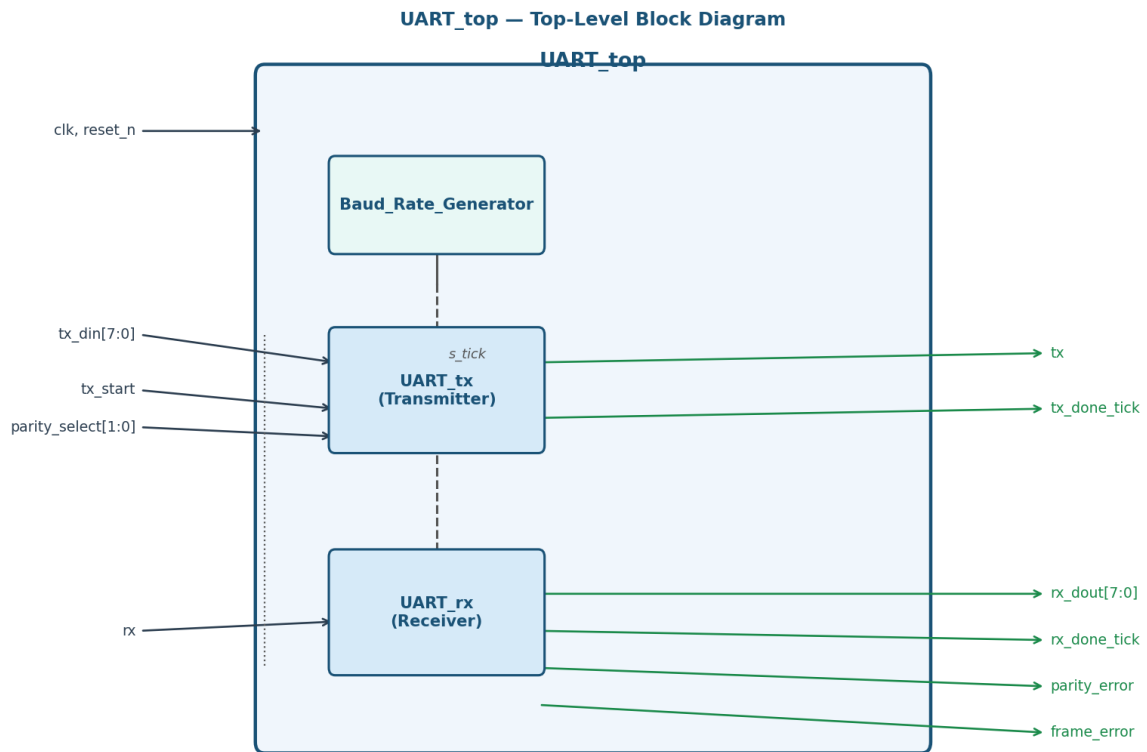


Figure 4. UART_top — Top-Level Block Diagram

5. Component Breakdown

5.1 Baud Rate Generator — Baud_Rate_Generator.v

A parameterized modulo-N counter. It divides the system clock to produce a single-cycle pulse (done) at $16\times$ the target baud rate. The counter increments on every rising clock edge while en is high. When the count reaches FINAL_VALUE, done is asserted for exactly one clock cycle and the counter resets to zero.

Parameters

Parameter	Default	Description
BITS	4	Width of the internal counter and the FINAL_VALUE input

Ports

Port	Direction	Description
clk	Input	System clock
reset_n	Input	Active-low asynchronous reset
en	Input	Counter enable — holds count when low
FINAL_VALUE	Input	[BITS-1:0] — counter rolls over at this value
done	Output	High for one clock cycle when count == FINAL_VALUE

5.2 UART Transmitter — UART_tx.v

Converts an 8-bit parallel word into a serial UART frame. Controlled by a 5-state Algorithmic State Machine with Datapath (ASMD). The parity bit is pre-computed in the idle state when tx_start is asserted — before data shifting begins — and stored in p_reg. This is necessary because the shift register b_reg is destroyed during the data state.

Parameters

Parameter	Default	Description
DBITS	8	Number of data bits per frame
SB_TICK	16	Stop-bit duration in s_ticks (16=1 stop bit, 32=2 stop bits)

Ports

Port	Direction	Description
clk	Input	System clock
reset_n	Input	Active-low asynchronous reset
s_tick	Input	$16\times$ baud rate sampling tick from BRG
tx_start	Input	Assert high for one cycle to initiate transmission
parity_select	Input	[1:0] — 2'b00=none, 2'b01=even, 2'b10=odd
tx_din	Input	[DBITS-1:0] — parallel data word to transmit
tx_done_tick	Output	Pulses high for one cycle when frame is complete
tx	Output	Serial data output line

State Machine

State	TX Line	Action
idle	1	Wait for tx_start. On assertion: load tx_din, pre-compute parity into p_reg.
start	0	Drive start bit low for 16 s_ticks, then advance to data.
data	b_reg[0]	Shift out data bits LSB-first, one bit per 16 s_ticks. After last bit advance to parity or stop.
parity	p_reg	Transmit pre-computed parity bit for 16 s_ticks, then advance to

		stop.
stop	1	Drive stop bit(s) for SB_TICK s_ticks. Assert tx_done_tick, return to idle.

5.3 UART Receiver — UART_rx.v

Recovers an 8-bit parallel word from an incoming serial UART frame using 16× oversampling. Includes a 2-FF input synchronizer for the raw rx input, parity checking, and frame error detection. All FSM logic operates on the synchronised signal rx_sync.

Parameters

Parameter	Default	Description
DBITS	8	Number of data bits per frame
SB_TICK	16	Stop-bit duration in s_ticks

Ports

Port	Direction	Description
clk	Input	System clock
reset_n	Input	Active-low asynchronous reset
rx	Input	Raw serial input — passes through 2-FF synchronizer
s_tick	Input	16× baud rate sampling tick from BRG
parity_select	Input	[1:0] — 2'b00=none, 2'b01=even, 2'b10=odd
rx_dout	Output	[DBITS-1:0] — recovered parallel data word
rx_done_tick	Output	Pulses high for one cycle when a complete frame is received
parity_error	Output	Pulses high (same cycle as rx_done_tick) on parity mismatch
frame_error	Output	Pulses high (same cycle as rx_done_tick) when stop bit is logic 0

State Machine

State	Advance Condition	Action
idle	rx_sync == 0	Start bit detected. Reset tick counter, advance to start.
start	s_tick && s_reg == 7	Confirm start bit at midpoint. Reset counters, advance to data.
data	s_tick && s_reg == 15	Sample rx_sync, right-shift into b_reg. Repeat for all DBITS. Advance to parity or stop.
parity	s_tick && s_reg == 15	Latch rx_sync into p_reg. Advance to stop.
stop	s_tick && s_reg == SB_TICK-1	Check stop bit and parity. Assert rx_done_tick, return to idle.

Error Detection Logic

Both error signals are combinational and valid on the same clock cycle as rx_done_tick:

```

frame_error = ~rx_sync                                (stop bit must be logic
1)
parity_error = (^b_reg) != p_reg                        [even parity]
parity_error = (~^b_reg) != p_reg                      [odd parity]

```

5.4 Top-Level Wrapper — UART_top.v

Integrates the three sub-modules. The BRG output s_tick is shared between TX and RX. The parity_select input is also shared, ensuring both sides always use the same parity configuration.

Parameters

Parameter	Default	Description
DBITS	8	Data bits per frame
SB_TICK	16	Stop-bit duration in s_ticks
BRG_BITS	4	Counter width of the Baud Rate Generator
BRG_FINAL	7	Counter limit — set per baud rate formula in Section 2

6. ASMD Charts

ASMD Chart of the UART Transmitter

The transmitter ASMD chart shows the five-state FSM and its datapath operations. State boxes (dashed blue borders) show Moore outputs. Decision diamonds show Mealy conditions. Operation boxes (green) show datapath register updates.

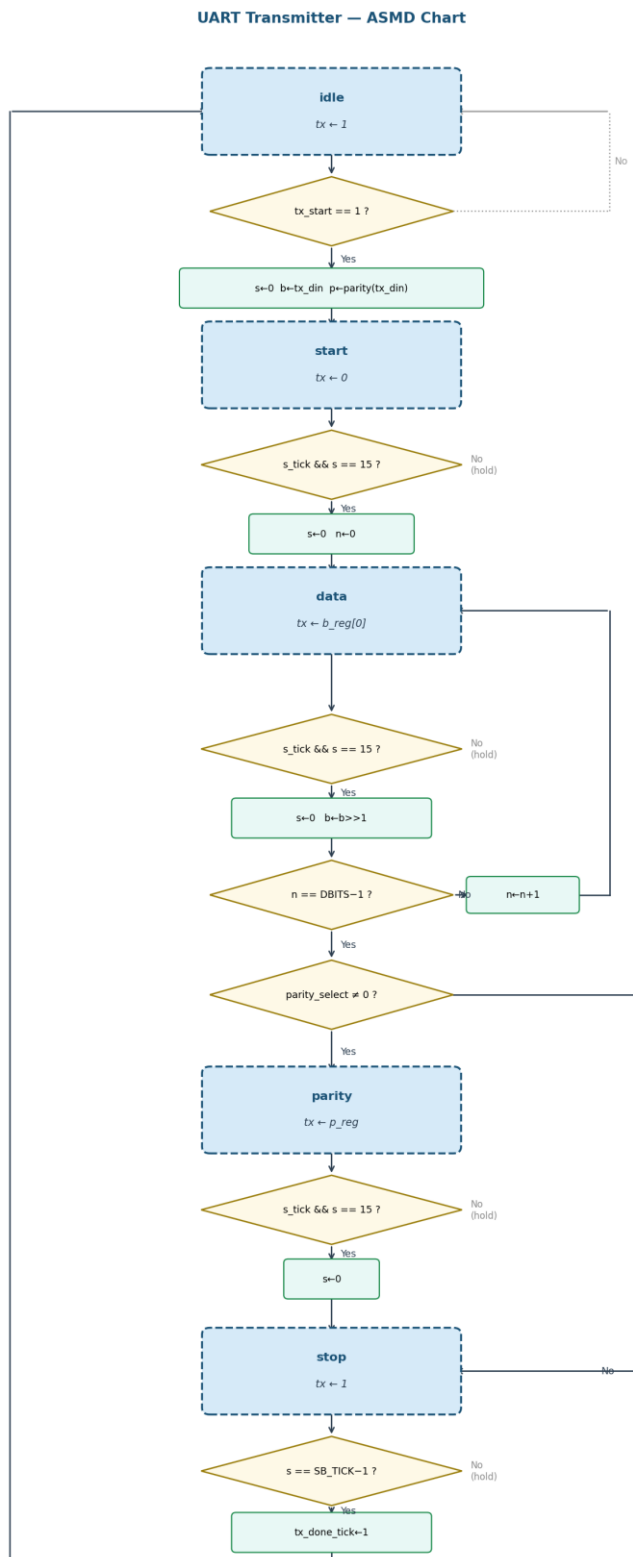


Figure 5. UART Transmitter ASMD Chart

ASMD Chart of the UART Receiver

The receiver ASMD chart mirrors the transmitter structure with the addition of the parity and frame error checks in the stop state. The rx_done_tick, parity_error, and frame_error signals are all asserted combinationally within the stop state when s_reg == SB_TICK-1.

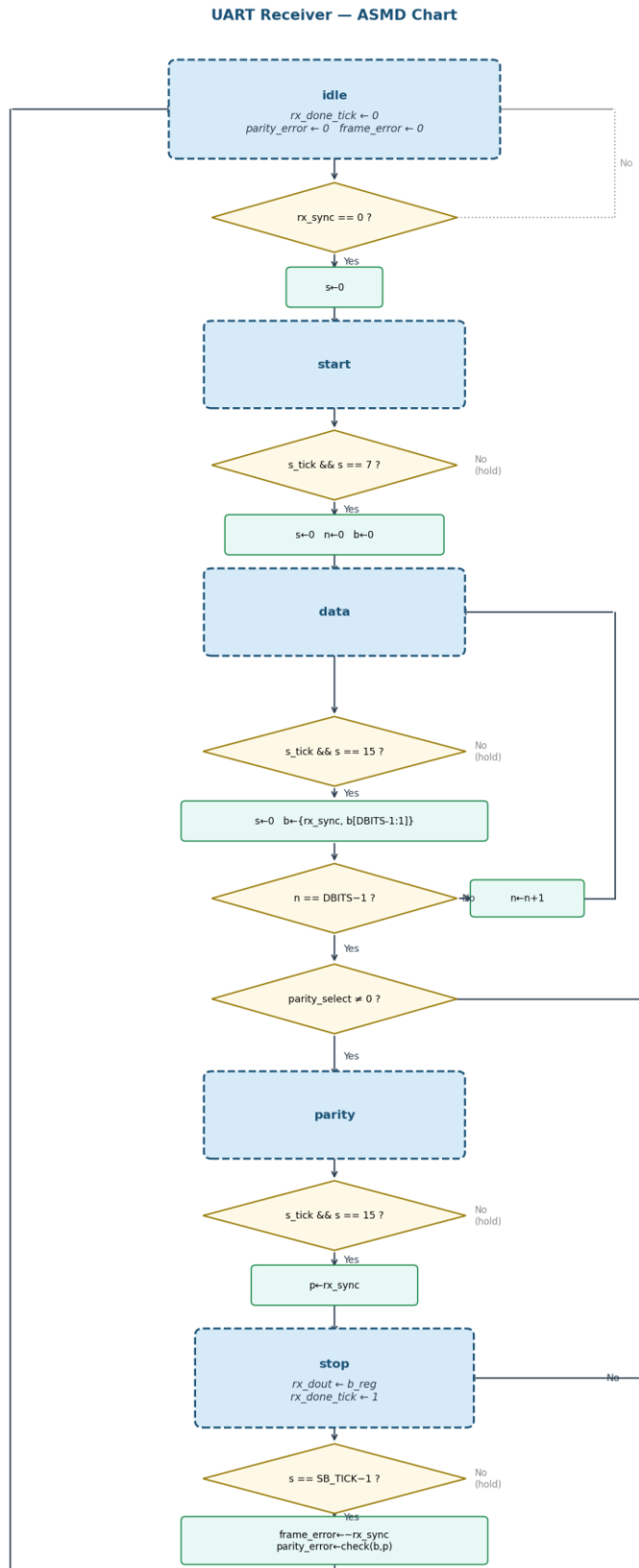


Figure 6. UART Receiver ASMD Chart

7. Testbenches & Simulation

7.1 Transmitter Testbench — UART_tx_tb.v

Tests the transmitter module in isolation. The Baud Rate Generator runs with `FINAL_VALUE = 7` (`s_tick` every 8 clock cycles) so that the individual bits on the tx line are clearly visible in the waveform. The testbench sends 7 bytes across three parity configurations using a `send_byte` task that asserts `tx_start` for one clock cycle and waits for `tx_done_tick`.

#	Data	Format	Description
1	0xE5	8N1	8 data bits, no parity, 1 stop bit
2	0x3C	8N1	8 data bits, no parity, 1 stop bit
3	0xF0	8N1	8 data bits, no parity, 1 stop bit
4	0xA5	8E1	8 data bits, even parity, 1 stop bit
5	0x55	8E1	8 data bits, even parity — alternating bits
6	0xAA	8O1	8 data bits, odd parity — alternating bits
7	0xFF	8O1	8 data bits, odd parity — all ones

The simulation waveform shows one complete byte transmission. The tx signal is driven low for the start bit, transitions between logic levels corresponding to each data bit (LSB first), then shows the parity bit and stop bit. Each level is held for exactly 16 `s_ticks` (one bit period).

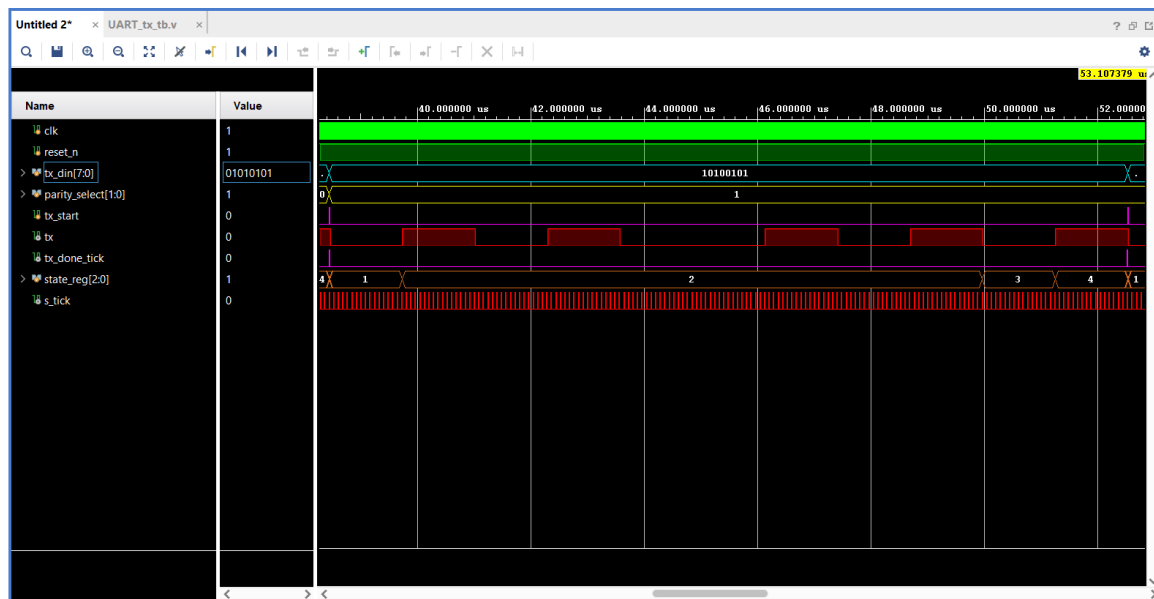


Figure 7. UART Transmitter Simulation Waveform — single byte transmission showing start bit, data bits, parity bit, and stop bit on the tx line

7.2 Full UART Loopback Testbench — UART_loopback_tb.v

Connects the transmitter's tx output directly to the receiver's rx input, testing the complete data path end-to-end. The BRG runs with FINAL_VALUE = 0 (s_tick fires every clock cycle) to maximise simulation speed. A send_and_verify task transmits each byte and automatically checks rx_dout, parity_error, and frame_error, printing PASS or FAIL to the console.

#	Data	Format	Expected Result
1	0xA5	8N1	rx_dout = 0xA5, no errors
2	0x55	8E1	rx_dout = 0x55, no errors
3	0xAA	8O1	rx_dout = 0xAA, no errors
4	0xF0	8N1	rx_dout = 0xF0, no errors
5	0x3C	8E1	rx_dout = 0x3C, no errors
6	0xE5	8O1	rx_dout = 0xE5, no errors

The simulation results confirm that all 6 test cases pass. The data recovered by the receiver (rx_dout) exactly matches the data loaded into the transmitter (tx_din) in every case. The parity_error and frame_error signals remain deasserted throughout, confirming correct parity generation and frame delimitation across all three parity configurations.

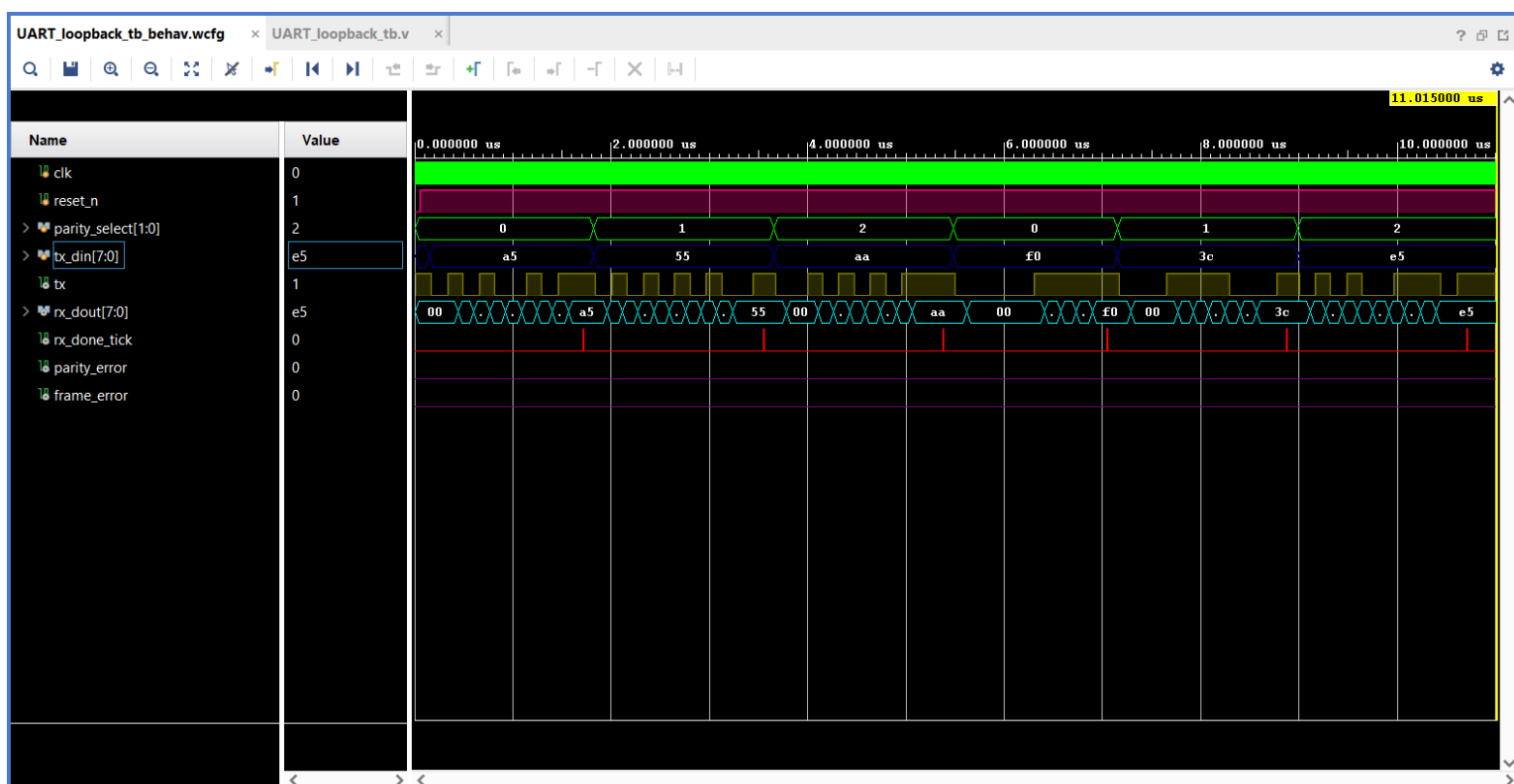


Figure 8. UART Loopback Simulation — all 6 test cases. tx_din and rx_dout match in every frame; parity_error and frame_error remain at 0.

